

AN
INTRODUCTION
TO
OPERATING
SYSTEM

BY RENIS,
AUTHOR OF DATA STRUCTURES AND ALGORITHMS.
This book is the same as the last series is just a way of taking notes and to
learn Operating Systems

Published by:
Just me taking notes with L^AT_EX
2026.

PREFACE

These are my personal notes on Operating Systems, a project I aim to complete by the end of this semester. For me, this is more than just a university subject; it is an opportunity to dive deep into the core textbooks rather than just following a syllabus. Coincidentally, I began my journey by learning C—the fundamental language of operating systems—through “Head First C” and Roger Ngo’s tutorials. My curiosity led me to study the Linux Kernel, understand the intricacies of Arch Linux, and experiment heavily within WSL.

This collection synthesizes knowledge from two primary sources: “Operating System Concepts” by Abraham Silberschatz and “Linux Kernel Development” (3rd Edition) by Robert Love. To faithfully follow Robert Love’s work, I set up a virtual environment running Ubuntu 10.04 with the 2.6 Linux Kernel. This experience made me feel like a true systems programmer—one who doesn’t just rely on the latest versions but appreciates the history and evolution of this beautiful technology. In these pages, you will find not just theoretical notes, but code implementations and practical experiments for every topic covered.

What Operating Systems Do

The operating system can be viewed as a **resource allocator**. A computer system has many resources that may be required to solve a problem: CPU time, memory space, storage space, I/O devices, and so on. The operating system acts as the manager of these resources, deciding how to allocate them to specific programs and users so that it can operate the computer system efficiently and fairly.

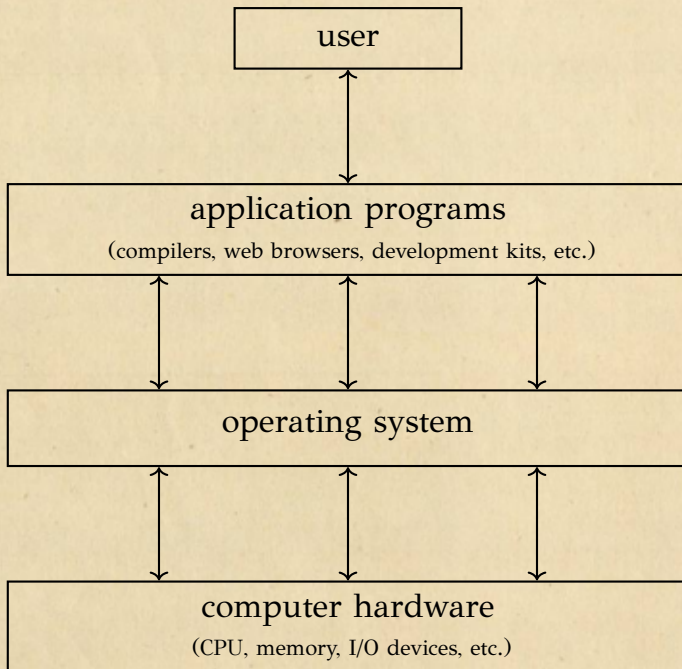


Figure 1.1 Abstract view of the components of a computer system.

Computer-System Organization

A modern general-purpose computer system consists of one or more CPU's and several device controllers connected through a common bus, providing access between the system components and shared memory. While the intricacies of inter-component communication will be detailed in subsequent chapters, the fundamental concept to grasp now is that every device is managed by a specific controller.

Each controller contains local buffer storage—conceptually similar to registers—and certain high-performance I/O devices may even feature dedicated DRAM chips. This architecture is designed primarily for asynchronous execution and speed; the CPU does not waste cycles waiting for relatively slow I/O operations. Instead, the CPU offloads the task to the device controller and continues executing other instructions. Once the controller completes its assigned task, it notifies the CPU, ensuring maximum system efficiency.

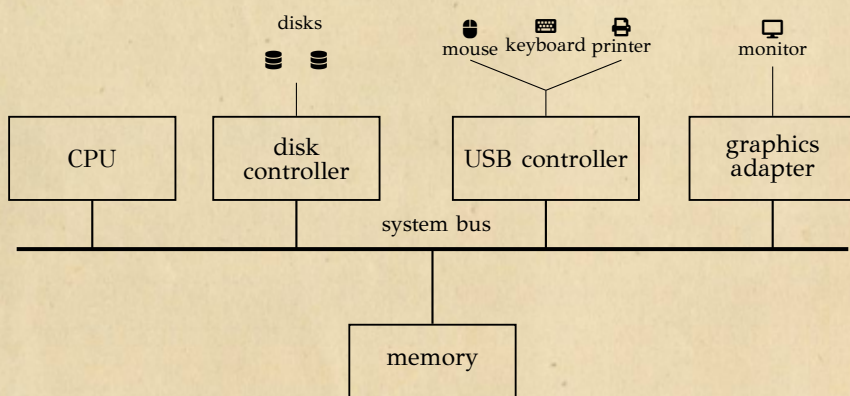


Figure 1.2 A typical PC computer system with integrated icons.

Interrupts

Modern operating systems utilize interrupts to manage asynchronous events and hardware faults effectively. At the hardware level, this is facilitated by an **interrupt-request line** which the CPU monitors after the execution of every instruction. Once the CPU detects that a controller has asserted a signal on this line, it immediately reads the interrupt number to index the **interrupt vector** and jumps to the corresponding service routine. To manage varying degrees of urgency, modern hardware implements a system of interrupt priorities. This priority-based architecture allows the CPU to defer low-priority interrupts without masking the entire system, making it possible for a high-priority interrupt to preempt the execution of a lower-priority handler. Ultimately, because interrupts are essential for time-sensitive processing, this streamlined mechanism of catching and dispatching interrupts is fundamental to achieving high system performance.

Storage Structure

The hierarchy of storage in computers is so efficient that we achieve a lot, from giant magnetic storage to small electronic RAM, and it is truly phenomenal. Nowadays, chips are so complex and powerful that we see the **Northbridge** and **Southbridge** being removed from the motherboard because we now use the **SoC (System on Chip)** architecture, where the CPU has integrated almost all motherboard components. This CPU also has internal storage with three types of cache: **L1** which is dedicated to only one core, **L2** which serves as a larger buffer, and **L3** which is shared across all cores inside the CPU. This

hierarchy follows the path $L1 \rightarrow L2 \rightarrow L3 \rightarrow \text{DRAM} \rightarrow \text{Secondary Memory}$, where starting from $L1$ we have very little memory but extremely high speed, while with **DRAM** and wider storage we achieve more space but less speed. All this storage is volatile, meaning when we shut down the system we lose everything in **RAM** and the CPU, so we need **Secondary Memory** like an **SSD** or **HDD** using electronic modules like **NVMe**, **FRAM**, or **SRAM** to save data long-term. Additionally, we have **EEPROM** which is a **Read-Only Memory (ROM)** found on the motherboard, which is different from when we write in `C char* c = "Hello";`. In that case, you cannot rewrite the string because the **GCC compiler** packages it into a specific part of the **.exe** file called the **.rodata** (Read-Only Data) section.

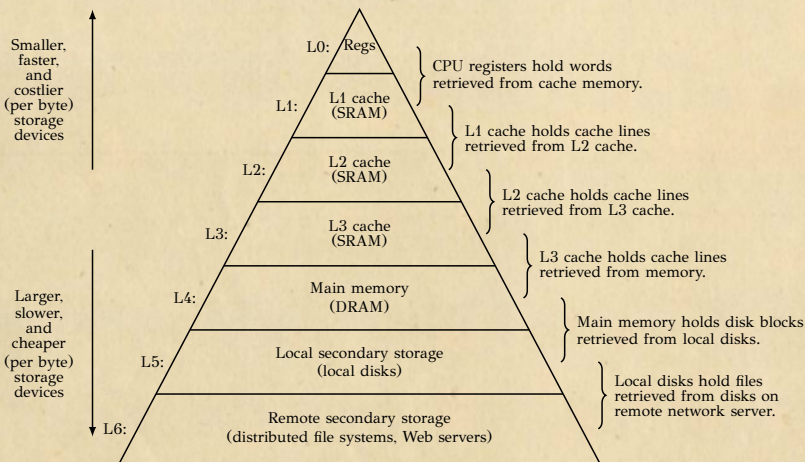
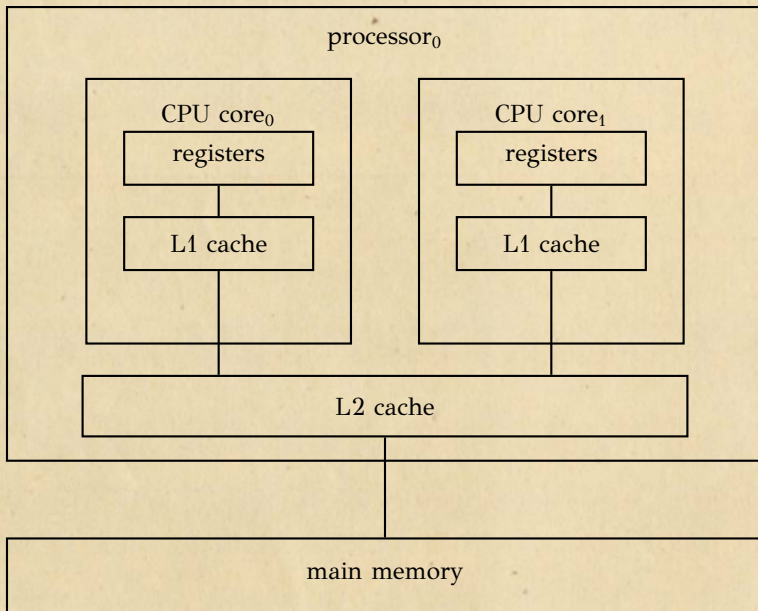
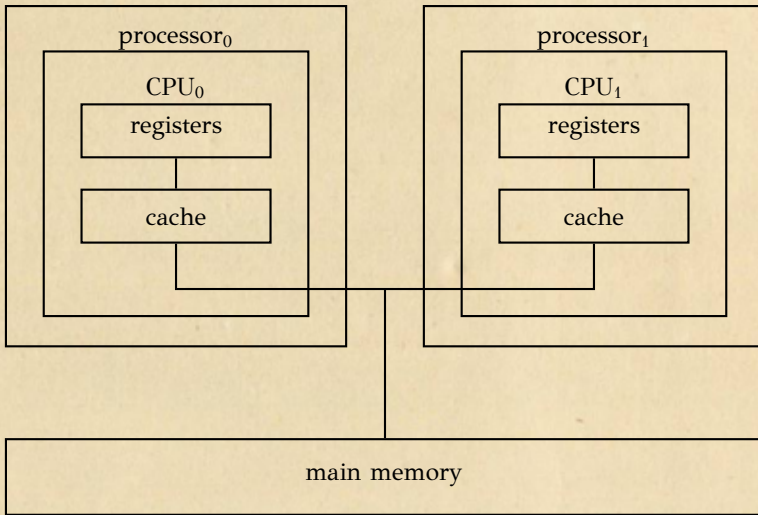


Figure 1.9 An example of a memory hierarchy.

Single-Processor & Multiprocessor System

While systems with a single traditional core are considered obsolete in contemporary computing, they laid the foundation for modern architectures by executing instructions using dedicated controllers for hardware components like disks and keyboards. However, the evolution of hardware led to the development of **multiprocessor systems** to achieve higher performance and **throughput**. It is important to note that performance does not increase linearly with the number of processors due to coordination overhead and **resource contention**. The most prevalent model today is **Symmetric Multiprocessing (SMP)**, where each peer CPU performs all tasks and shares physical memory via the system bus, while maintaining its own private registers and **local cache**.

Modern designs have further evolved into **multicore systems**, where multiple computing cores are integrated onto a single **chip** to enhance communication speed and significantly reduce power consumption, a factor especially critical for mobile devices. In these architectures, each core retains its own set of registers and a private **L1 cache**, while typically sharing a **L2 cache** located on the same chip. This arrangement allows the Operating System to distribute processes and resources dynamically, ensuring efficient **load balancing** across all processing units to maximize system responsiveness.



There are numerous fundamental concepts to explore, such as **user mode**, **kernel mode**, and the intricate ways a CPU manages concurrent tasks through **multiprocessing** and **scheduling**. However, as this section serves merely as an overview, we are not attempting to replicate the entire textbook. Instead, our objective is to distill the core essence of these topics into a format that is as concise and clear as possible. We will not delve into advanced fields such as **Virtual Machine Management (VMM)** or system security at this stage; these will be addressed in later sections. For now, Chapter 1 remains a high-level introduction to the foundational architecture of operating systems.